ML Unit-3 Notes:

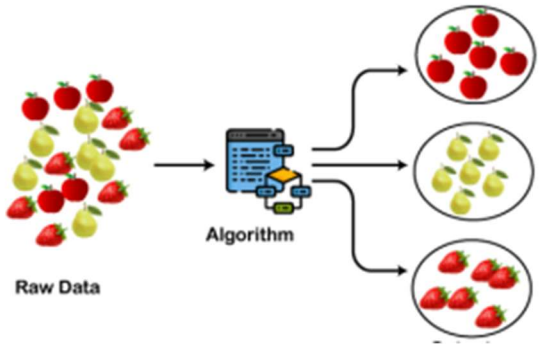
CLUSTERING AND DIMENSIONALITY REDUCTION

Topics to cover

- Unsupervised learning: K-means, Instance Based Learning
- KNN, Gaussian mixture models and Expectation maximization.
- Dimensionality Reduction
- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Latent Variable Models (LVM)
- Latent Dirichlet Allocation (LDA)
- Independent Component Analysis (ICA)

Unsupervised learning: K-means, Instance Based Learning:

Clustering:



ADVANCED TAXONOMY OF CLUSTER ANALYSIS AND ALGORITHMIC ARCHITECTURE

Clustering is an unsupervised learning process that automatically groups unlabeled data so that objects in the same group are highly similar while objects in different groups are highly dissimilar.

- Cluster: A collection of data objects that belong to the same class because they share a common trait.
- Unsupervised Learning: The process of discovering the natural structure or grouping in a dataset without the need for prior training or labels.

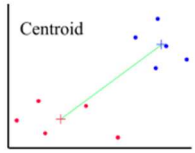
- Intra-class Similarity: A quality metric where the variation between data points within a single cluster is minimized.
- Inter-class Similarity: A quality metric where the separation between different clusters is maximized.
- Distance-Based Clustering: A method where membership is determined by the geometrical closeness of objects.
- Euclidean Metric: The standard mathematical measure used to calculate the distance D between two points p and q, defined by the formula

$$D = \sum_{i=1}^{k} (P_i - q_i)^2$$

DESIRABLE PROPERTIES OF A CLUSTERING ALGORITHM

- Scalability: The algorithm must remain efficient in terms of time and space as the volume of data increases.
- Data Type Versatility: The ability to process and group different types of data effectively.
- Minimal Domain Knowledge: The system should require very few input parameters from the user to function.
- Interpretability: The final clusters must be usable and provide meaningful insights into the data structure.
- In this case we easily identify the 4 clusters into which the data can be divided; the similarity criterion is distance: two or more objects belong to the same cluster if they are “close” according to a given distance (in this case geometrical distance) This is called distance –based clustering.

CLUSTER CENTROID LOGIC AND ARCHITECTURE



- Centroid Definition: The centroid is a central point in a cluster where its parameter values are the means of the values of every point assigned to that cluster.
- Membership Determination: Algorithms evaluate the distance between individual data points and these centroids to decide which group a point belongs to.

- Functional Sequence:
    1. Initialize the process by selecting or calculating central points for potential groups.
    2. Calculate the distance between each raw data point and the available centroids using the Euclidean metric.
    3. Assign each data point to the cluster with the closest centroid.
    4. Update the centroid by calculating the new mean of all points currently assigned to the group.
    5. Repeat until the centroids stabilize and no further reassignments occur.
    6. Output a statistical description of the centroids and the number of components in each resulting cluster.

CLUSTERING TECHNIQUES TYPES

- Partitioning Methods: These divide a dataset into a pre-defined number of distinct, non-overlapping groups.
- Hierarchical Methods: These create a tree-like structure of nested clusters.
- Density-Based Methods: These group points based on the spatial density of data, allowing for discovery of arbitrary shapes.
- Exclusive Clustering: A rigid method where every data point is assigned to exactly one specific cluster.
- Overlapping Clustering: A flexible method where a single data point may have membership in multiple clusters.
- Probabilistic/Model-Based Clustering: Methods like Gaussian Mixture Models where each point has a probability of belonging to various clusters based on statistical distributions.

THE "TRAP" LIST

- The Best Criterion Myth: There is no absolute "best" clustering method; the quality of the output depends entirely on the user's specific goals and the similarity measures selected.
- Centroid Misidentification: A centroid is a mathematical average (mean) of cluster members and is not necessarily a point that existed in the original dataset.

- Overlapping Failure: Exclusive algorithms like K-means will fail to represent data accurately if the natural groups are soft or overlapping.

CONCENTRATED SUMMARY

Clustering is an unsupervised technique used to partition large datasets into meaningful subclasses by maximizing intra-class similarity and minimizing inter-class similarity. High-quality algorithms must be scalable, interpretable, and capable of handling diverse data types with minimal user intervention. The core functional unit is the cluster centroid—the mean of all points in a group—which uses distance metrics like the Euclidean formula to determine group membership. Major techniques include partitioning, hierarchical, and density-based methods, which can be implemented as exclusive, overlapping, or probabilistic models depending on the data structure.

K-Means Clustering:

K-MEANS CLUSTERING ARCHITECTURE AND OPTIMIZATION STRATEGY

K-means clustering is a method that sorts a messy pile of unlabeled data into a specific number of groups by making sure every point is as close as possible to a central "home base" called a centroid.

- Unsupervised Learning: K-means is an unsupervised algorithm, meaning it discovers patterns in data without any prior training or pre-defined labels.
- The "K" Factor: The variable K defines the exact number of non-overlapping clusters the algorithm will create.
- Centroid-Based Logic: Each cluster is defined by a central point (centroid), and the algorithm's primary goal is to minimize the total distance between data points and these centroids.
- Iterative Process: The algorithm repeats its calculations over and over until it finds the best possible grouping for the data.
- Data Membership: In this model, every data point is assigned to exactly one cluster based on shared properties and proximity.

ALGORITHM AND STEP-BY-STEP ARCHITECTURE

- Task 1 (Center Identification): The algorithm uses an iterative process to find the best physical coordinates for K center points.

- Task 2 (Point Assignment): The algorithm assigns every individual data point to its nearest center to form distinct groups.
- Functional Sequence:
    1. Initialization: Decide on the number of clusters K.
    2. Random Selection: Choose K random points to serve as the initial centroids; these points do not need to be part of the actual dataset.
    3. Initial Grouping: Calculate the distance from every data point to these K centroids and assign each point to the closest one.
    4. Centroid Update: Look at all the points in a cluster, find their average (center of gravity), and move the centroid to this new location.
    5. Reassignment Loop: Re-calculate the distance of every point to the updated centroids and move any point that is now closer to a different center.
    6. Convergence: Repeat the update and reassignment steps until no more points change groups and the centroids stop moving.

Steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

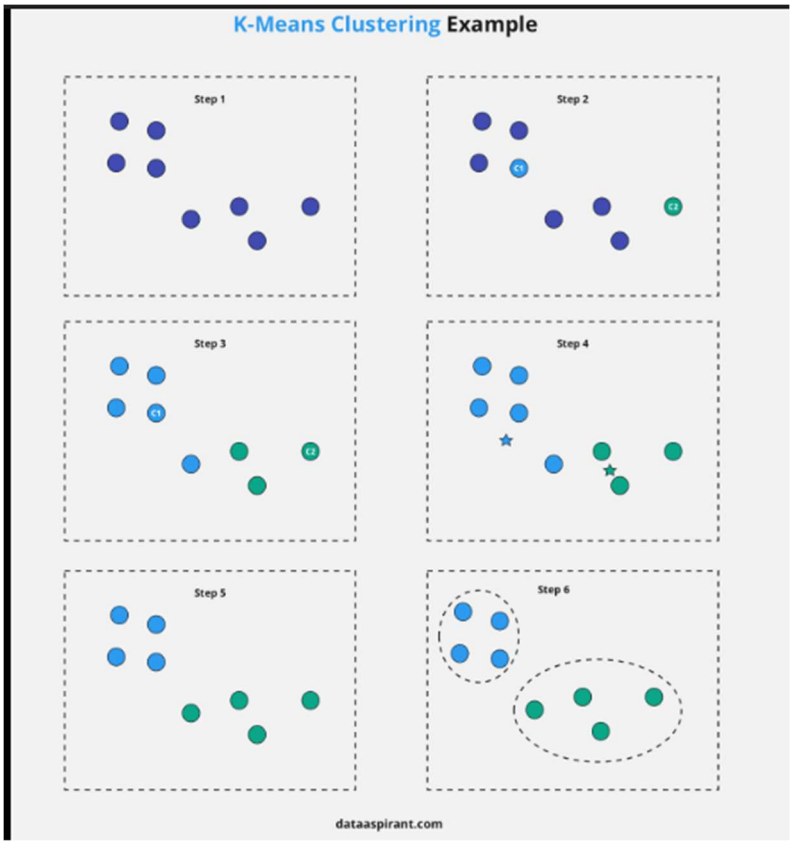
Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.



THE ELBOW METHOD AND OPTIMAL "K" SELECTION

- Selection Challenge: The performance of the model depends on choosing the right K, but there is no automatic way for the algorithm to know the best number on its own.
- WCSS (Within Cluster Sum of Squares): This is the mathematical metric used to measure how "compact" or "tight" the clusters are.

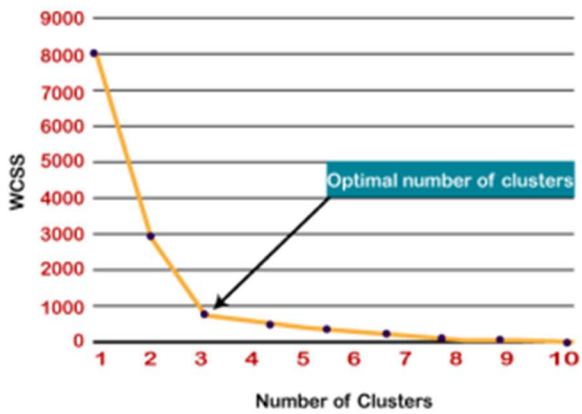
$$WCSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i\ C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i\ C_2)^2 + \sum_{P_i \text{ in CLuster3}} \text{distance}(P_i\ C_3)^2$$

- WCSS Calculation: It is the sum of the squared distances between every point in a cluster and that cluster's centroid.
- The Elbow Workflow:
    1. Run the K-means algorithm multiple times using different values for K, usually ranging from 1 to 10.

2. Calculate the WCSS for every single K value tested.
3. Plot the results on a graph where the x-axis is K and the y-axis is the WCSS.
4. Identify the "elbow" point, which is the sharp bend in the curve where the WCSS stops dropping rapidly.
5. Selection: The K value at this "elbow" is the optimal balance between cluster count and data accuracy.

THE "TRAP" LIST

- The Randomness Trap: Because the algorithm starts with random centroids, different starting positions can lead to different final clusters.
- Hard Membership: K-means is an "exclusive" clustering method, meaning a point cannot belong to two clusters even if it sits exactly in the middle.
- The Fixed K Requirement: Unlike other methods, you must define K before you start; the algorithm cannot "discover" that it needs more or fewer groups while running.



CONCENTRATED SUMMARY

K-means is a centroid-based unsupervised algorithm that partitions unlabeled data into K distinct clusters by minimizing the sum of distances between points and their group centers. The process is highly iterative, involving the initialization of random centroids, point assignment based on proximity, and the continuous updating of centroids based on the mean of the cluster members until convergence is reached. To ensure the model is effective, users employ the Elbow Method, which identifies the optimal K value by finding the point were increasing the number of clusters no longer significantly reduces internal variation (WCSS).

K-NN

SUPERVISED SIMILARITY ARCHITECTURE: K-NEAREST NEIGHBORS (KNN)

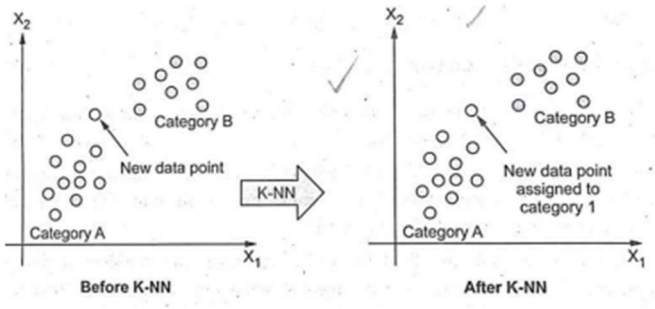
Core Logic KNN is a supervised learning method that classifies a new data point by identifying its most similar neighbors in a stored dataset and assigning it to the most common category among them.

Exhaustive Breakdown

- Supervised Learning approach: The algorithm relies on pre-labeled data to classify new instances.
- Similarity-based Logic: New cases are placed into categories most similar to existing classes based on feature proximity.
- Non-parametric Algorithm: It makes zero assumptions about the underlying distribution of the data.
- Lazy Learner: KNN does not learn a model during a training phase; instead, it stores the dataset and performs all computations during classification.
- Classification and Regression: While primarily used for classification problems, it can also handle regression tasks.
- Memory Dependency: The algorithm must store the entire dataset to categorize new data points.
- The "K" Parameter: This represents the specific number of neighbors used to make a prediction.
- Distance Metric: Typically utilizes Euclidean distance to measure how "close" points are in the feature space.
- Feature Mapping: Uses coordinates (like X1 and X2) to plot data and determine class boundaries.

Why We Need KNN

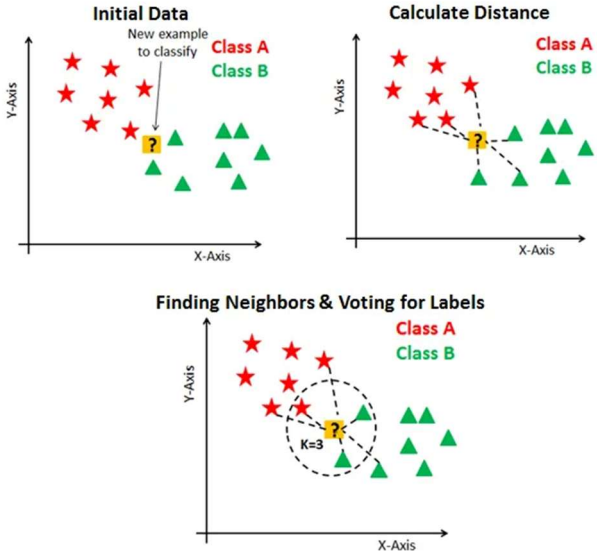
- Category Identification: It is essential for determining which group a specific data point belongs to when it falls between known categories.
- Similarity Measurement: It solves complex identification problems, such as distinguishing between a cat and a dog based on shared physical features.



- Simple Implementation: It provides a straightforward way to discover classes without complex model training.

Step-by-Step Architecture: KNN Functional Sequence

1. Neighbor Selection: Define the value of K, which is the number of neighbors to be considered (e.g., K=5).
2. Distance Calculation: Compute the Euclidean distance between the new data point and every existing point in the dataset.
3. Neighbor Identification: Select the K points that have the smallest calculated distances to the new data point.
4. Category Counting: Tally the number of data points belonging to each category among those K neighbors.
5. Majority Assignment: Assign the new data point to the category with the highest neighbor count (e.g., if 3 neighbors are Category A and 2 are Category B, assign to A).



Comparison: K-means vs. KNN

- Learning Type: K-means is an unsupervised machine learning algorithm, whereas KNN is a supervised learning algorithm.
- Purpose: K-means is used exclusively for clustering unlabeled data, while KNN is used for classification and sometimes regression.
- Learning Style: K-means is an "eager learner" that processes data to find patterns early, while KNN is a "lazy learner" that waits until a query is made.
- Data Requirements: K-means requires unlabeled data to gather and group points, but KNN requires labeled data to predict test samples.
- Definition of "K": In K-means, K refers to the number of clusters the algorithm creates; in KNN, K refers to the number of neighbors used to classify a point.

The "Trap" List

- Computational Cost: Because KNN is a lazy learner, it becomes extremely slow as the dataset size increases.
- K-Value Sensitivity: Choosing a K that is too small can lead to noise affecting the results, while a K that is too large might include points from unrelated classes.
- Distance Metric Bias: Euclidean distance calculation

$$D = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

- assumes all features are on the same scale.
- Storage Requirements: The model requires significant memory because it must store every training instance to function.

Concentrated Summary KNN is a lazy, non-parametric supervised learning algorithm that classifies data based on the similarity of new instances to stored, labeled examples. By calculating the Euclidean distance between a query point and its neighbors, it uses a majority vote among the K-nearest points to determine the correct class. Unlike the unsupervised K-means clustering algorithm, KNN relies on pre-defined labels and performs all logic at the time of classification rather than through a prior training phase.

Gaussian Mixture models and Expectation Maximization

Gaussian Mixture Models are a "soft" clustering probabilistic model that assumes all data points are generated from a mixture of a finite number of Gaussian distributions with unknown parameters.

Functional Logic of GMM

- **Soft Clustering:** Unlike K-means which rigidly assigns a point to one cluster, GMM allows each point to probabilistically "belong" to all clusters.
- **Distribution Assumption:** It assumes the underlying data follows a sum of scaled and shifted normal (Gaussian) distributions.
- **Probabilistic Factor:** The model determines the probability that each specific data point belongs to a certain Gaussian distribution.
- **Real-World Use:** Applications include signal processing, language identification, song genre classification, and customer churn analysis.

The Expectation-Maximization (EM) Process

The EM algorithm is the mathematical engine used to estimate the parameters of a GMM when the data has hidden or unobserved (latent) variables.

Functional Sequence

- **Initial Step:** Define observable variables X and hidden/latent variables Z.
- **E-Step (Expectation):** * Compute the expected values of the latent variables using the current parameter settings.
    - This step treats latent variables as if they were observed to calculate the likelihood.
- **M-Step (Maximization):** * Re-evaluate parameters by maximizing the log-likelihood found in the E-step.
    - This determines if new data points can be effectively added to the components.
- **Convergence:** The algorithm alternates between E and M steps until the parameters stabilize or a convergence criterion is met.

The "Trap" List: High-Yield Nuances

- **Convergence Issues:** While the initial rate of convergence is good, it can become "excruciatingly slow" as it approaches local optima.
- **The Overfitting Risk:** EM works best when the fraction of missing info is small; high dimensionality can dramatically slow down the E-step.
- **Superiority Over Substitution:** EM is more effective than mean or regression substitution because those methods generate biased estimates and underestimate standard errors.
- **Local vs. Global Maxima:** Convergence is guaranteed to a local maximum, or a global maximum only if the likelihood function is unimodal.

High-Density Summary

GMM provides a flexible, probabilistic framework for clustering where data points maintain fractional membership across multiple Gaussian distributions. The EM algorithm serves as the iterative optimizer for this model, alternating between estimating hidden data (E-step) and updating model parameters (M-step) until the best statistical fit is achieved. It is the primary tool for handling latent variable problems and managing missing data in complex datasets.

Dimensionality reduction

Dimensionality Dynamics and Reduction Strategies

The **Curse of Dimensionality** refers to the phenomenon where increasing the number of features (dimensions) leads to a decrease in machine learning performance because the required number of training examples increases exponentially with each added dimension.

The Curse of Dimensionality

- **Data Sparsity:** As dimensions (D) increase, data points become increasingly sparse; for example, if 9 samples are dense in 1D, you need 9^3 = 729 samples to maintain the same density in 3D.

- **Computational Complexity:** Most machine learning methods have a complexity of at least O(n^2) or involve d^2 parameters in a covariance matrix, leading to massive computational overhead.
- **Overfitting Risk:** With too many parameters (d) and too few samples (n), models tend to overfit; it is often better to assume features are uncorrelated to avoid this.

Optimal Number of Features for Classification

- **The Paradox:** Theoretically, more features should improve performance, but in practice, adding features beyond a certain point increases classification error.
- **The Sweet Spot:** For every fixed sample size n, there is an "optimal number of features" that results in the minimum possible classification error.

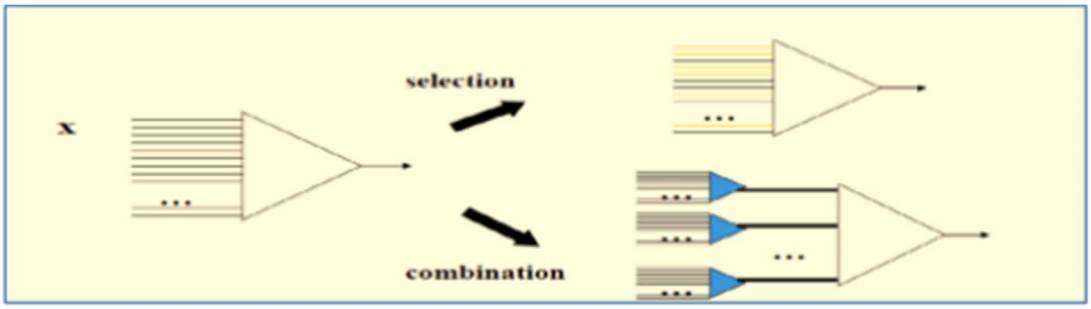
Why Dimensionality Reduction?

- **Efficiency:** It reduces time complexity (less computation) and space complexity (less storage).
- **Robustness:** Simpler models are more robust and less prone to overfitting, especially on small datasets.
- **Visualization:** Reducing data to 2 or 3 dimensions allows for the visualization of structures, groups, and outliers.
- **Data Cleaning:** It removes features that bear little useful information or "noise," leaving only the "essence" of the data.

The Process and Criterion for Reduction

- **Mechanism:** High-dimensional points are projected into a lower-dimensional space while preserving distances and the "essence" of the data as much as possible.
- **Combination vs. Selection:** Dimensions are reduced by either selecting a subset of existing variables or transforming/combining multiple variables into new ones.
- **Geometric/Information Criteria:** Reduction must capture the "signals" or variations in data; variables that are highly correlated or dependent can be combined.

Feature Selection vs. Feature Reduction



- **Feature Selection:** Only a smaller subset of the original features is chosen based on an "interestingness" measure; the rest are discarded.
- **Feature Reduction:** All original features are used but are linearly or non-linearly transformed into a smaller set of new features.

Linear Transformation for Feature Reduction

In a linear setting, the original high-dimensional data x is mapped to a lower-dimensional space y using a transformation matrix G.

Formula:

$$y = G^T x$$

Where $x \in \mathbb{R}^p$ (original), $y \in \mathbb{R}^d$ (reduced), and $d \ll p$. +1

Applications of Feature Reduction

- **Face and Image Recognition:** Reducing pixel-heavy data into manageable components.
- **Handwritten Digit Recognition:** Extracting only relevant patterns from a set of strokes.
- **Text Mining and image retrieval:** Sorting through massive datasets for key Information.
- **Bioinformatics:** Microarray data analysis and protein classification.

High-Density Summary

Dimensionality reduction is the process of mapping high-dimensional data into a lower-dimensional space while preserving its "essence" to combat the **Curse of Dimensionality**. It strikes a balance between performance and complexity, finding the **Optimal Number of Features** that results in the lowest classification error. By using **Feature Selection** (choosing the best) or **Feature Reduction** (transforming them all), models become faster, more efficient, and less likely to overfit.

PCA

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms a large set of correlated variables into a smaller set of uncorrelated variables called **Principal Components** while preserving as much variance (information) as possible.

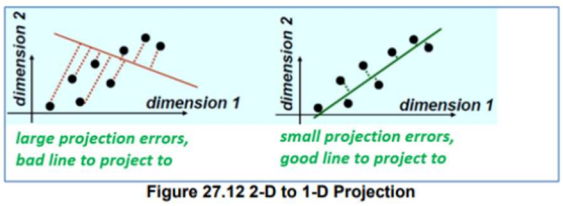


Figure 27.12 2-D to 1-D Projection

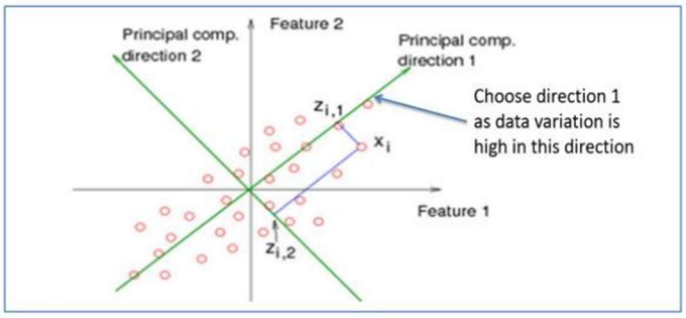
Core Logic of PCA

- **Information Preservation:** It seeks the most accurate data representation in a lower-dimensional space.
- **Variance Maximization:** PCA projects data along directions where the data varies the most.
- **Orthogonality:** It chooses new directions that are orthogonal (at 90 degrees) to each other to minimize correlation between features.
- **Error Minimization:** The best line for projection is the one where the projection error (distance from points to the line) is minimum.

Step-by-Step Architecture of PCA

To reduce a d-dimensional dataset to k dimensions, follow this functional sequence:

1. **Mean Centering:** Calculate the sample mean of all data vectors and subtract this mean from each data point.
2. **Covariance/Scatter Matrix:** Compute the scatter matrix (or symmetric covariance matrix) estimated from the samples.
3. **Eigen-Decomposition:** Calculate the eigenvectors and eigenvalues of the scatter matrix.
4. **Selection:** Identify and select the k eigenvectors corresponding to the k largest eigenvalues.
5. **Projection:** Use these K eigenvectors as columns to create a transformation matrix and project the original data into the new k-dimensional subspace.



Practical Use and Benefits

- **Feature Pre-processing:** PCA is used to clean and prepare features before actual classification is performed.
- **Redundancy Elimination:** If two features are highly correlated, one dimension can be dropped without losing significant information.
- **Noise Reduction:** By dropping the least significant dimensions (those with very small eigenvalues), much of the noise in the data is eliminated.
- **Pattern Identification:** Reducing dimensionality enables the identification of the strongest, most dominant patterns in a dataset.

The "Trap" List: Critical Nuances

- **Classification Limitation:** PCA is designed for **data representation**, not classification; it only considers feature properties, not class labels.
- **Variance Misalignment:** If the direction of maximum variance is not the direction that separates classes, PCA will fail to help with classification.
- **Orthogonal Basis:** PCA can be thought of as finding a new basis by rotating the old axes until the directions of maximum variance are found.

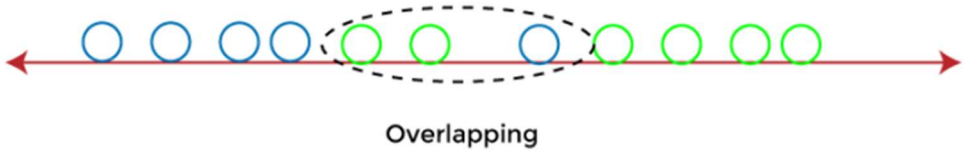
High-Density Summary

PCA is a powerful tool for simplifying complex datasets by rotating data axes to align with the directions of highest variance. By focusing on these "Principal Components" and discarding dimensions with minimal variance, it reduces the dataset size, removes noise, and handles redundant features. While it is excellent for summarizing data structure, it is an unsupervised method that does not account for class differences during the reduction process.

Linear Discriminant Analysis (LDA):

Linear Discriminant Analysis (LDA)

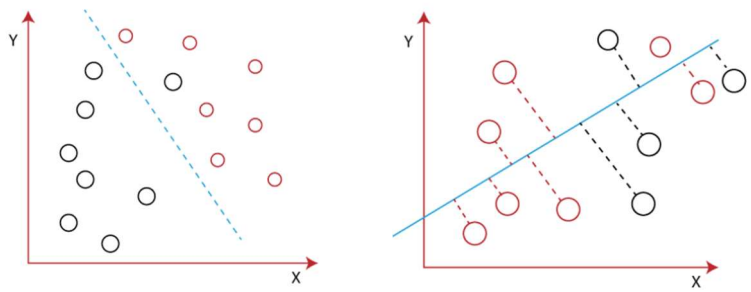
Linear Discriminant Analysis is a supervised dimensionality reduction technique used to project high-dimensional features into a lower-dimensional space while maximizing the separability between two or more classes. Unlike PCA, which is unsupervised and focuses on variance, LDA is specifically designed for classification problems.



Core Logic and the Overlapping Problem

- **The Problem:** When classes have multiple features, classifying them using only a single feature often leads to "overlapping," where data points from different categories occupy the same space on a graph.

- **The Solution:** To overcome overlapping, LDA increases the number of features or, more effectively, reduces the data from a 2D or 3D plane into a 1-dimensional plane in a way that maximizes class separation.
- **Separation Criteria:** LDA creates a new axis based on two primary conditions:
    1. **Maximize the distance** between the means of the different classes.
    2. **Minimize the variance** (scatter) within each individual class.



Functional Sequence of the LDA Model

To achieve efficient classification, the LDA model follows these operational principles:

- **Feature Transformation:** It uses an X-Y axis to create a new axis by projecting data onto a straight line that completely separates the classes.
- **Multi-Class Handling:** While Logistic Regression is limited to binary (two-class) problems, LDA efficiently handles multiple classes.
- **Pre-processing:** It acts as a pre-processing step to reduce the number of features, which significantly lowers computing costs.

Extensions and Variations

LDA has several specialized versions to handle different data complexities:

- **Quadratic Discriminant Analysis (QDA):** Used for multiple input variables where each class has its own estimate of variance.
- **Flexible Discriminant Analysis (FDA):** Designed for non-linear groups of inputs, such as those using splines.
- **Regularized LDA:** Uses regularization in the covariance estimate to moderate the influence of different variables on the model.

Real-World Applications

- **Face Recognition:** Used in "Fisherfaces" to minimize features (pixel values) into a manageable number before classification.
- **Medical Classification:** Classifies patient disease severity (mild, moderate, or severe) based on health parameters to help doctors adjust treatment pace.
- **Customer Identification:** Identifies specific groups of customers in shopping malls who are most likely to purchase a particular product.
- **Robotics and Learning:** Used to train robots to simulate human speech by building groups based on pitches, frequencies, and tunes.
- **Decision Making:** Provides predicted results for binary choices, such as "will you buy this product" (buying vs. not buying).

Drawbacks of LDA

- **Shared Means:** LDA fails if the means of the distributions are shared between classes, as it cannot create an axis that makes them linearly separable.
- **Linear Limitation:** It is fundamentally a linear technique; for complex, non-linear problems, a non-linear discriminant analysis is required.
- **Data Assumptions:** It assumes the input variables follow a Gaussian (normal) distribution and have the same variance, requiring data standardization and outlier removal for best results.

High-Density Summary

LDA is a supervised strategist for classification that solves the "overlapping" problem by projecting data onto a new axis that simultaneously pulls class means apart and pushes individual class data closer together. It is more versatile than Logistic Regression for multi-class tasks and serves as a critical pre-processing tool in fields ranging from medical diagnosis to facial recognition. However, its effectiveness relies on classes having distinct means and following a normal distribution.

LDA

Latent Variable Models (LVM) and Latent Dirichlet Allocation (LDA)

Latent Variable Models aim to model the probability distribution of data by using hidden (latent) variables to "explain" the data in a simpler, lower-dimensional way.

Core Logic of LVM

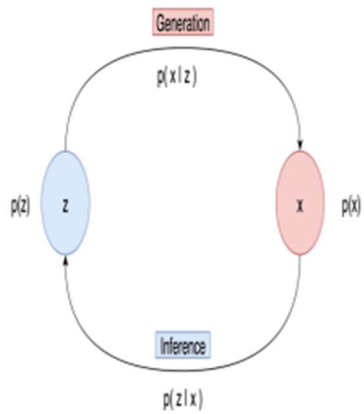
- **Mapping:** Data points x, following a distribution $p(x)$, are mapped into latent variables z that follow a distribution $p(z)$.
- **No Labels:** These models are unsupervised and do not use any form of labels y.
- **Dimensionality:** Latent variables represent a transformation of data points into a continuous lower-dimensional space.

The Five Basic Probability Terms

1. **Prior Distribution p(z):** Models the behavior and assumptions of the latent variables.
2. **Likelihood p(x|z):** Defines the mapping from latent variables back to the observed data points.
3. **Joint Distribution p(x,z):** The product of the likelihood and the prior (p(x|z)p(z)); it essentially describes the entire model.
4. **Marginal Distribution p(x):** The distribution of the original data; it tells us how likely it is to generate a specific data point and is the ultimate goal of the model.
5. **Posterior Distribution p(z|x):** Describes which latent variables are likely to be produced by a specific observed data point.

Process Architecture: Generation vs. Inference

- **Generation:** The process of computing a data point x from a latent variable z (moving from latent space to actual data).
  - **Sequence:** Sample $z \sim p(z)$, then sample $x \sim p(x|z)$.
- **Inference:** The inverse process of finding the latent variable z from a data point x.
  - **Sequence:** Sample $x \sim p(x)$, then sample $z \sim p(z|x)$.



Training via Maximum Likelihood

- **Goal:** Estimate parameters θ so that the probability distribution best fits the observed data.
- **Optimization:** Since it cannot be solved analytically, iterative approaches like **Gradient Descent** are used.
- **Likelihood Function:** Measures the "goodness of fit" of the model to the data sample.

$$\theta^{ML} = arg \max_{\theta} \sum_{i=1} log p_\theta(x_i)$$

As you can tell, it is a standard optimization problem. It can't be solved analytically so we use an

iterative approach such as gradient descent. Once it's solved, we can derive the model

parameters θθ which effectively model the desired probability distribution.

Nonetheless, in order to apply gradient descent, we need to calculate the gradient of the marginal

log-likelihood function. Using simple calculus and the Bayes rule, we can prove that:

$$\nabla log p_\theta(x) = \int p_\theta(z|x) \nabla_\theta log p_\theta(x, z) dz$$

Latent Dirichlet Allocation (LDA - Topic Modeling)

LDA is a specific generative model used for topic modeling, where documents are seen as mixtures of latent topics.

- **Core Assumption:** Documents with similar topics use similar groups of words.
- **Generative Setup:** * Each document is represented as a K-dimensional vector of topic distributions.
    - Each topic is a distribution over the vocabulary words.
- **The Dirichlet Distribution:** This is the "prior" used in LDA; it acts as a probability density for the multinomial parameters.
    - **Visualization:** Often shown as a triangle (simplex) where corners represent topics and points inside represent word-topic relations.

The "Trap" List: High-Yield Nuances

- **Inference Difficulty:** The fundamental problem in LDA is computing the posterior of hidden variables given the document; this is often mathematically complex.
- **Parameter Interconnection:** All distributions in LVM are interconnected via **Bayes' Rule**.
- **Iterative Requirement:** Maximum likelihood for these models usually requires iterative optimization because analytical solutions are unfeasible.

High-Density Summary

Latent Variable Models simplify complex data by assuming it is generated by hidden factors (z). By defining the relationship between observed data (x) and these factors through

Likelihood and **Prior** distributions, we can perform **Generation** (creating data) or **Inference** (understanding data). **LDA** is the most famous application of this, allowing us to discover hidden topics in text by assuming words are generated from latent thematic distributions.

ICA

Independent Component Analysis (ICA) Overview

Independent Component Analysis (ICA) is a powerful unsupervised learning algorithm used to separate a multivariate signal into additive, mutually independent subcomponents.

Core Concept: The Cocktail Party Problem

- **Scenario**: Imagine a cocktail party where multiple people speak simultaneously, making it hard to follow one conversation.
- **Technical Setup**: If you record with two microphones, you get two mixed signals.
- **Goal**: The objective of ICA is to recover the original, unknown source signals by separating this mixed data into mutually independent dimensions.

Functional Logic of ICA vs. PCA

- **PCA (Principal Component Analysis)**: PCA works by maximizing variances using eigenvectors and eigenvalues to identify the principal direction of a relationship.
- **The Failure of PCA**: PCA fails for datasets with more than one main direction, such as a "cross pattern" of mixed signals.
- **The Strength of ICA**: ICA focuses on identifying **independent components** rather than just the main directions.

The 3-Step ICA Algorithm Sequence

To find the original signals (s), the algorithm performs inverse operations through a series of steps ($A^{-1} = U^T \Sigma^{-1} V$):

- **Step 1: Rotation (Estimate U^T)**: Finds the angle with maximal variance to rotate the data, which is derived from the primary direction determined through PCA. +1
- **Step 2: Stretching (Estimate Σ^{-1})**: Scales the principal components by stretching the figure using the variances (σ_1 and σ_2) of the data. +1
- **Step 3: Independent Rotation (Estimate V)**: Rotates signals around angle Φ using independence and kurtosis assumptions to rebuild the original dimensions of the signals.

The "Trap" List: Key Limitations

- **Linear Constraint**: ICA can only separate sources that have been **linearly mixed**.
- **Gaussian Limitation**: It **cannot** separate sources that are perfectly Gaussian-distributed, as this would prevent the third step of the algorithm from functioning.
- **Amplitude Ambiguity**: The algorithm cannot recover the exact amplitude of the source activity, which often results in estimated signals appearing inverted.

Real-World Applications

- **Signal Processing**: Separating multivariate signals into subcomponents.
- **Neuroimaging**: Brain imaging and other biometric data analysis.
- **Finance**: Predictive analysis for stock markets and other complex correlated datasets.
- **Computer Vision**: Powering face recognition applications.

High-Density Summary

ICA is a powerful tool for discovering hidden structures in data by separating mixed, correlated information into its original, independent sources. By using a three-stage inverse operation process of rotation and scaling based on non-Gaussianity, it extracts relevant insights that would otherwise be lost in a sea of correlations. It is widely used in signal processing, finance, and medical fields.